

LAPORAN PRAKTIKUM 9
STRUKTUR DATA
IMPLEMENTASI *ABSTRACT DATA TYPE (ADT)*
SEARCHING (BFS dan DFS)



Disusun Oleh:

Naysha Aprilia Rizki

2511531014

Dosen Pengampu:

Wahyudi. Dr. S.T.M.T

Asisten Praktikum:

Muhammad Galid Averro

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul “**Laporan Praktikum Struktur Data: Implementasi Abstract Data Type (ADT) Searching (BFS dan DFS) Menggunakan Bahasa Java**” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah Struktur Data. Adapun tujuan dari praktikum ini adalah untuk memahami konsep **Abstract Data Type (ADT)** serta mengimplementasikan algoritma pencarian (*searching*) menggunakan metode **Breadth First Search (BFS)** dan **Depth First Search (DFS)** dalam bahasa pemrograman Java. Selain itu, praktikum ini juga bertujuan agar mahasiswa mampu memahami cara kerja proses pencarian pada struktur data berbasis graf, penggunaan node dan edge, serta perbedaan mekanisme penelusuran antara algoritma BFS dan DFS.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai struktur data, khususnya penerapan **Abstract Data Type (ADT)** dan algoritma **Searching BFS dan DFS** dalam pemrograman Java.

Padang, 11 Juni 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengolahan dan pencarian data secara cepat dan efisien merupakan salah satu aspek yang sangat penting. Salah satu konsep yang digunakan dalam pengelolaan data adalah **Abstract Data Type (ADT)**. ADT merupakan konsep yang mendefinisikan struktur data berdasarkan data yang disimpan serta operasi yang dapat dilakukan terhadap data tersebut tanpa memperhatikan bagaimana implementasinya secara detail.

Salah satu penerapan ADT dalam struktur data adalah proses pencarian (*searching*). Searching digunakan untuk menemukan data tertentu dari sekumpulan data berdasarkan kriteria yang telah ditentukan. Pada struktur data seperti graf (*graph*), proses pencarian dapat dilakukan menggunakan beberapa algoritma, di antaranya **Breadth First Search (BFS)** dan **Depth First Search (DFS)**.

Breadth First Search (BFS) merupakan algoritma pencarian yang melakukan penelusuran secara melebar, yaitu mengunjungi node berdasarkan tingkat atau level terdekat terlebih dahulu sebelum berpindah ke level berikutnya. Sedangkan **Depth First Search (DFS)** melakukan pencarian secara mendalam dengan mengunjungi satu jalur hingga mencapai node terakhir sebelum kembali (*backtracking*) untuk mencari jalur lainnya.

Setiap algoritma searching memiliki cara kerja dan karakteristik yang berbeda. BFS biasanya menggunakan struktur data queue untuk menyimpan node yang akan dikunjungi, sedangkan DFS menggunakan stack atau metode rekursif dalam proses penelusurannya. Pemahaman terhadap kedua algoritma ini sangat diperlukan agar dapat menentukan metode pencarian yang sesuai berdasarkan kebutuhan dan kondisi data.

Pada praktikum ini dilakukan implementasi **Abstract Data Type (ADT) Searching menggunakan algoritma BFS dan DFS** dengan bahasa pemrograman Java. Praktikum ini bertujuan untuk memahami konsep ADT, struktur data graf, serta bagaimana proses penelusuran data dilakukan menggunakan algoritma pencarian BFS dan DFS.

1.2 Tujuan

1. Memahami konsep dasar **Abstract Data Type (ADT)** dalam struktur data.
2. Memahami cara kerja algoritma pencarian **Breadth First Search (BFS)** dan **Depth First Search (DFS)**.
3. Mengimplementasikan algoritma BFS dan DFS menggunakan bahasa pemrograman Java.
4. Memahami penggunaan struktur data graf dalam proses pencarian data.
5. Melatih kemampuan pemrograman dalam mengelola dan melakukan pencarian data secara efisien menggunakan struktur data.

1.3 Manfaat

1. Menambah pemahaman mengenai konsep **Abstract Data Type (ADT)** dan penerapannya dalam struktur data.
2. Membantu memahami perbedaan cara kerja algoritma **BFS dan DFS** dalam melakukan pencarian data.
3. Melatih kemampuan implementasi algoritma searching menggunakan bahasa pemrograman Java.
4. Memberikan pengalaman dalam mengolah struktur data graf dan proses penelusuran node.
5. Menjadi dasar pembelajaran untuk memahami algoritma pencarian dan struktur data yang lebih kompleks.

BAB II

PEMBAHASAN

Dalam pemrograman, struktur data merupakan cara untuk menyimpan dan mengatur data agar dapat digunakan secara efektif. Salah satu konsep penting dalam struktur data adalah **Abstract Data Type (ADT)**. ADT merupakan konsep yang mendefinisikan data beserta operasi yang dapat dilakukan terhadap data tersebut tanpa menjelaskan secara detail bagaimana proses implementasinya. Dengan menggunakan ADT, program dapat dibuat lebih terstruktur dan mudah dikembangkan.

Pada praktikum ini dilakukan implementasi **Abstract Data Type (ADT) Searching** menggunakan algoritma pencarian **Breadth First Search (BFS)** dan **Depth First Search (DFS)**. Kedua algoritma tersebut digunakan untuk melakukan proses penelusuran data pada struktur **Graph** dan **Tree**. Implementasi dilakukan menggunakan bahasa pemrograman Java dengan beberapa class, yaitu **Node_2511531014**, **BTree_2511531014**, **GraphTraversal_2511531014**, dan **BTDriver_2511531014**.

Class **Node_2511531014** digunakan sebagai representasi node atau simpul pada struktur data. Node berfungsi sebagai tempat penyimpanan data serta hubungan antar node melalui pointer atau referensi. Pada struktur tree, node digunakan untuk menghubungkan bagian parent dengan child, sedangkan pada graph node digunakan untuk menyimpan hubungan antar simpul.

1. Node_2511531014

Class **Node_2511531014** digunakan untuk membuat objek node yang menjadi dasar dari struktur tree maupun graph. Setiap node menyimpan data tertentu serta memiliki hubungan dengan node lainnya.

Node memiliki fungsi penting karena menjadi penghubung antar elemen dalam struktur data. Dengan adanya node, proses penambahan, pencarian, dan penelusuran data dapat dilakukan menggunakan hubungan antar pointer.

2. BTree_2511531014

BTree merupakan implementasi struktur data **Binary Tree** yang terdiri dari kumpulan node yang saling terhubung. Setiap node pada binary tree memiliki maksimal dua anak, yaitu child kiri (*left*) dan child kanan (*right*).

Pada implementasi ini, struktur tree digunakan sebagai media untuk melakukan proses penelusuran menggunakan algoritma pencarian. Setiap node akan dikunjungi berdasarkan aturan algoritma yang digunakan.

Proses penelusuran pada tree dapat dilakukan dengan beberapa metode, salah satunya adalah:

1. Mengunjungi node awal sebagai titik mulai pencarian.
2. Mengecek data pada node tersebut.
3. Melanjutkan pencarian ke node anak sesuai metode traversal.
4. Mengulangi proses hingga data ditemukan atau seluruh node telah diperiksa.

3. BTDriver_2511531014

Class **BTDriver_2511531014** digunakan sebagai class utama untuk menjalankan program. Pada class ini dilakukan pembuatan objek tree atau graph, memasukkan data, serta menjalankan proses pencarian menggunakan algoritma BFS dan DFS.

Melalui class driver, pengguna dapat melihat hasil proses traversal dan memahami urutan node yang dikunjungi oleh masing-masing algoritma.

Berdasarkan implementasi yang telah dilakukan, dapat disimpulkan bahwa konsep **Abstract Data Type (ADT)** membantu dalam mengatur struktur data menjadi lebih terorganisir. Algoritma **BFS dan DFS** memiliki cara kerja yang berbeda dalam melakukan pencarian, di mana BFS melakukan pencarian secara melebar menggunakan Queue, sedangkan DFS melakukan pencarian secara mendalam menggunakan Stack atau rekursif. Implementasi menggunakan Java memberikan pemahaman mengenai hubungan antar node, proses traversal, serta penerapan algoritma searching pada struktur data Tree dan Graph.

4. GraphTraversal_2511531014

Class **GraphTraversal_2511531014** digunakan untuk melakukan proses traversal atau penelusuran pada struktur graph menggunakan algoritma BFS dan DFS.

Graph merupakan struktur data yang terdiri dari kumpulan vertex (node) dan edge (hubungan antar node). Setiap node dapat memiliki hubungan dengan beberapa node lainnya.

Breadth First Search (BFS)

Breadth First Search (BFS) merupakan algoritma pencarian yang melakukan penelusuran secara melebar berdasarkan tingkat atau jarak node dari titik awal.

Langkah kerja BFS:

1. Menentukan node awal pencarian.
2. Menyimpan node awal ke dalam Queue.
3. Mengambil node dari Queue dan menandainya sebagai sudah dikunjungi.
4. Mengunjungi node tetangga yang belum dikunjungi.
5. Memasukkan node tersebut ke dalam Queue.
6. Mengulangi proses hingga semua node telah dikunjungi.

Kelebihan BFS adalah mampu menemukan jalur terpendek pada graf tanpa bobot. Namun, BFS membutuhkan memori lebih besar karena harus menyimpan daftar node yang akan dikunjungi.

Depth First Search (DFS)

Depth First Search (DFS) merupakan algoritma pencarian yang melakukan penelusuran secara mendalam. DFS akan mengunjungi satu jalur hingga mencapai node terakhir sebelum kembali mencari jalur lainnya.

Langkah kerja DFS:

1. Menentukan node awal.
2. Mengunjungi node tersebut dan menandainya.
3. Melanjutkan pencarian ke node berikutnya yang belum dikunjungi.
4. Melakukan proses secara rekursif hingga tidak ada node yang dapat dikunjungi.
5. Kembali ke node sebelumnya untuk mencari cabang lainnya.

Kelebihan DFS adalah penggunaan memori yang lebih kecil dibandingkan BFS karena hanya menyimpan jalur yang sedang diproses.

1. Langkah Kerja

1.1. Node_2511531014

- Kode Prrogram

```

1 package pekan9_2511531014;
2
3 public class Node_2511531014 {
4
5     int data_1014;
6     Node_2511531014 left_1014;
7     Node_2511531014 right_1014;
8
9     public Node_2511531014(int data_1014) {
10         this.data_1014 = data_1014;
11         left_1014 = null;
12         right_1014 = null;
13     }
14     public void setLeft_1014(Node_2511531014 node_1014) {
15         if (left_1014 == null)
16             left_1014 = node_1014;
17     }
18     public void setRight_1014(Node_2511531014 node_1014) {
19         if (right_1014 == null)
20             right_1014 = node_1014;
21     }
22     public Node_2511531014 getLeft_1014() {
23         return left_1014;
24     }
25     public Node_2511531014 getRight_1014() {
26         return right_1014;
27     }
28     public int getData_1014() {
29         return data_1014;
30     }
31     public void setData_1014(int data_1014) {
32         this.data_1014 = data_1014;
33     }
34     void printPreorder_1014(Node_2511531014 node_1014) {
35         if (node_1014 == null)
36             return;
37         System.out.print(node_1014.data_1014 + " ");
38         printPreorder_1014(node_1014.left_1014);
39         printPreorder_1014(node_1014.right_1014);

```

```

40     }
41     void printPostorder_1014(Node_2511531014 node_1014) {
42         if (node_1014 == null)
43             return;
44         printPostorder_1014(node_1014.left_1014);
45         printPostorder_1014(node_1014.right_1014);
46         System.out.print(node_1014.data_1014 + " ");
47     }
48     void printInorder_1014(Node_2511531014 node_1014) {
49         if (node_1014 == null)
50             return;
51         printInorder_1014(node_1014.left_1014);
52         System.out.print(node_1014.data_1014 + " ");
53         printInorder_1014(node_1014.right_1014);
54     }
55     public String print_1014() {
56         return this.print_1014("", true, "");
57     }
58     public String print_1014(String prefix_1014, boolean isTail_1014, String sb_1014) {
59         if (right_1014 != null) {
60             right_1014.print_1014( prefix_1014 + (isTail_1014 ? "|" : " "), false, sb_1014);
61         }
62         System.out.println( prefix_1014 + (isTail_1014 ? "\\-- " : "/-- ") + data_1014);
63         if (left_1014 != null) {
64             left_1014.print_1014( prefix_1014 + (isTail_1014 ? " " : "| "), true, sb_1014);
65         }
66         return sb_1014;
67     }
68 }

```

- Pembahasan

Program ini dibuat dengan mendefinisikan class **Node_2511531014** yang digunakan untuk mengimplementasikan struktur data **Binary Tree**. Class ini berfungsi sebagai representasi sebuah node yang memiliki data serta dua pointer, yaitu pointer ke node kiri (*left*) dan pointer ke node kanan (*right*).

1. Class ini memiliki atribut **data_1014**, **left_1014**, dan **right_1014**. Variabel **data_1014** digunakan untuk menyimpan nilai pada node, sedangkan **left_1014** dan **right_1014** digunakan sebagai referensi atau pointer yang menghubungkan node dengan anak kiri dan anak kanan.
2. Pada constructor **Node_2511531014(int data_1014)** digunakan untuk membuat objek node baru. Constructor menerima parameter berupa nilai data yang akan disimpan pada node. Pada saat node dibuat, pointer

left_1014 dan **right_1014** diatur bernilai null karena node tersebut belum memiliki anak.

3. Method **setLeft_1014(Node_2511531014 node_1014)** digunakan untuk menambahkan node baru sebagai anak kiri dari node saat ini. Proses penambahan dilakukan dengan pengecekan kondisi, yaitu node kiri hanya akan diisi apabila nilai **left_1014** masih kosong atau bernilai null.
4. Method **setRight_1014(Node_2511531014 node_1014)** memiliki fungsi yang sama seperti **setLeft**, tetapi digunakan untuk menambahkan node baru pada bagian kanan. Method ini akan menghubungkan node saat ini dengan node anak kanan.
5. Method **getLeft_1014()** dan **getRight_1014()** digunakan untuk mengambil atau mengakses node anak kiri dan anak kanan. Kedua method ini membantu proses penelusuran (*traversal*) pada struktur Binary Tree.
6. Method **getData_1014()** digunakan untuk mengambil nilai data yang tersimpan pada node, sedangkan method **setData_1014(int data_1014)** digunakan untuk mengubah nilai data yang terdapat pada node.
7. Method **printPreorder_1014(Node_2511531014 node_1014)** digunakan untuk melakukan traversal Preorder pada Binary Tree. Pada metode ini, proses dimulai dengan mengunjungi node utama (*root*), kemudian dilanjutkan ke node bagian kiri, lalu ke node bagian kanan. Urutan prosesnya adalah $\text{Root} \rightarrow \text{Left} \rightarrow \text{Right}$.
8. Method **printPostorder_1014(Node_2511531014 node_1014)** digunakan untuk melakukan traversal Postorder. Pada metode ini, node bagian kiri dikunjungi terlebih dahulu, kemudian node bagian kanan, dan terakhir node utama (*root*). Urutan prosesnya adalah $\text{Left} \rightarrow \text{Right} \rightarrow \text{Root}$.
9. Method **printInorder_1014(Node_2511531014 node_1014)** digunakan untuk melakukan traversal Inorder. Proses dimulai dari node kiri, kemudian node utama, lalu node kanan. Urutan prosesnya adalah $\text{Left} \rightarrow \text{Root} \rightarrow \text{Right}$.
10. Method **print_1014()** digunakan untuk menampilkan bentuk struktur Binary Tree dalam bentuk visual. Method ini memanggil method **print_1014(String prefix_1014, boolean isTail_1014, String sb_1014)** untuk mengatur tampilan cabang kiri dan kanan dari tree agar lebih mudah dipahami.
11. Pada method **print_1014(String prefix_1014, boolean isTail_1014, String sb_1014)** dilakukan proses pencetakan struktur tree secara rekursif. Node kanan akan ditampilkan terlebih dahulu, kemudian node utama, dan terakhir

node kiri sehingga bentuk hubungan antar node dapat terlihat seperti struktur pohon.

12. Secara keseluruhan, class **Node_2511531014** digunakan sebagai dasar pembentukan struktur data **Binary Tree** dengan memanfaatkan konsep pointer atau referensi antar node. Class ini mendukung proses penambahan node, pengambilan data, perubahan data, serta berbagai metode traversal seperti Preorder, Inorder, dan Postorder dalam pemrograman Java.

1.2. BTree_2511531014

- Kode Program

```
1 package pekan9_2511531014;
2
3 public class BTree_2511531014 {
4
5     private Node_2511531014 root_1014;
6     private Node_2511531014 currentNode_1014;
7
8     public BTree_2511531014() {
9         root_1014 = null;
10    }
11    public boolean search_1014(int data_1014) {
12        return search_1014(root_1014, data_1014);
13    }
14    private boolean search_1014(Node_2511531014 node_1014,
15                                int data_1014) {
16        if (node_1014.getData_1014() == data_1014)
17            return true;
18        if (node_1014.getLeft_1014() != null)
19            if (search_1014(node_1014.getLeft_1014(), data_1014))
20                return true;
21        if (node_1014.getRight_1014() != null)
22            if (search_1014(node_1014.getRight_1014(), data_1014))
23                return true;
24
25        return false;
26    }
27    public void printInorder_1014() {
28        root_1014.printInorder_1014(root_1014);
29    }
30    public void printPreOrder_1014() {
31        root_1014.printPreorder_1014(root_1014);
32    }
33    public void printPostOrder_1014() {
34        root_1014.printPostorder_1014(root_1014);
35    }
36    public Node_2511531014 getRoot_1014() {
37        return root_1014;
38    }
39    public boolean isEmpty_1014() {
40        return root_1014 == null;
41    }
42    public int countNodes_1014() {
43        return countNodes_1014(root_1014);
44    }
45    private int countNodes_1014(Node_2511531014 node_1014) {
46        int count_1014 = 1;
47        if (node_1014 == null) {
48            return 0;
49        } else {
50            count_1014 += countNodes_1014(node_1014.getLeft_1014());
51            count_1014 += countNodes_1014(node_1014.getRight_1014());
52            return count_1014;
53        }
54    }
55    public void print_1014() {
56        root_1014.print_1014();
57    }
58    public Node_2511531014 getCurrent_1014() {
59        return currentNode_1014;
60    }
61    public void setCurrent_1014(Node_2511531014 node_1014) {
62        this.currentNode_1014 = node_1014;
63    }
64    public void setRoot_1014(Node_2511531014 root_1014) {
65        this.root_1014 = root_1014;
66    }
67 }
```

- Pembahasan

Program ini dibuat dengan mendefinisikan class **BTree_2511531014** yang digunakan untuk mengimplementasikan struktur data **Binary Tree** serta operasi pencarian dan penelusuran (*traversal*) pada tree. Class ini berfungsi untuk mengelola node-node pada Binary Tree yang telah dibuat menggunakan class **Node_2511531014**.

1. Class ini memiliki atribut **root_1014** dan **currentNode_1014**. Variabel **root_1014** digunakan sebagai penunjuk node awal atau akar (*root*) dari Binary Tree, sedangkan **currentNode_1014** digunakan untuk menyimpan node yang sedang aktif atau sedang diproses.
2. Pada constructor **BTree_2511531014()**, nilai awal **root_1014** diatur menjadi null. Hal ini menunjukkan bahwa ketika objek Binary Tree pertama kali dibuat, kondisi tree masih kosong dan belum memiliki node.
3. Method **search_1014(int data_1014)** digunakan untuk melakukan proses pencarian data pada Binary Tree. Method ini memanggil method pencarian lain yang bersifat private dengan mengirimkan node root sebagai titik awal pencarian.
4. Method **search_1014(Node_2511531014 node_1014, int data_1014)** merupakan method rekursif yang digunakan untuk menelusuri seluruh node pada tree. Program akan mengecek nilai data pada node saat ini. Jika data ditemukan, method akan mengembalikan nilai true.
5. Pada proses pencarian, program terlebih dahulu mengecek node utama, kemudian melakukan pencarian ke bagian kiri menggunakan **getLeft_1014()**. Jika data belum ditemukan, pencarian dilanjutkan ke bagian kanan menggunakan **getRight_1014()**. Proses ini terus dilakukan secara rekursif hingga data ditemukan atau seluruh node selesai diperiksa.
6. Method **printInorder_1014()** digunakan untuk menampilkan isi Binary Tree menggunakan metode traversal Inorder. Method ini memanggil fungsi **printInorder_1014()** pada class Node dengan urutan penelusuran Left → Root → Right.
7. Method **printPreOrder_1014()** digunakan untuk menampilkan data dengan metode traversal Preorder. Proses penelusuran dilakukan dengan urutan Root → Left → Right, yaitu mengunjungi node utama terlebih dahulu kemudian node anak kiri dan kanan.
8. Method **printPostOrder_1014()** digunakan untuk menampilkan data menggunakan metode traversal Postorder. Urutan penelusurannya adalah Left → Right → Root, yaitu mengunjungi seluruh anak node terlebih dahulu sebelum kembali ke node utama.

9. Method **getRoot_1014()** digunakan untuk mengambil node root dari Binary Tree. Sedangkan method **setRoot_1014(Node_2511531014 root_1014)** digunakan untuk menentukan atau mengubah node yang menjadi root pada tree.
10. Method **isEmpty_1014()** digunakan untuk mengecek apakah Binary Tree memiliki data atau masih kosong. Jika nilai **root_1014** bernilai null, maka method akan mengembalikan nilai true yang berarti tree belum memiliki node.
11. Method **countNodes_1014()** digunakan untuk menghitung jumlah node yang terdapat pada Binary Tree. Method ini memanggil fungsi rekursif **countNodes_1014(Node_2511531014 node_1014)** yang menghitung jumlah node dengan menjumlahkan node utama, node kiri, dan node kanan.
12. Pada method **countNodes_1014(Node_2511531014 node_1014)**, jika node bernilai null maka proses berhenti dan mengembalikan nilai 0. Jika node tersedia, program akan menghitung node tersebut kemudian melakukan pemanggilan rekursif ke bagian kiri dan kanan hingga seluruh node selesai dihitung.
13. Method **print_1014()** digunakan untuk menampilkan bentuk struktur Binary Tree secara visual. Method ini memanggil method **print_1014()** yang terdapat pada class Node untuk menampilkan hubungan antar node dalam bentuk pohon.
14. Method **getCurrent_1014()** digunakan untuk mengambil node yang sedang aktif, sedangkan method **setCurrent_1014(Node_2511531014 node_1014)** digunakan untuk mengatur node aktif sesuai kebutuhan program.
15. Secara keseluruhan, class **BTree_2511531014** digunakan untuk mengelola struktur data Binary Tree dengan menyediakan operasi pencarian (*searching*), pengecekan kondisi tree, perhitungan jumlah node, serta berbagai metode traversal seperti Inorder, Preorder, dan Postorder. Class ini menerapkan konsep Abstract Data Type (ADT) karena menyediakan operasi terhadap struktur data tanpa memperlihatkan detail implementasi node secara langsung.

1.3. BTreeDriver_2511531014

- Kode Program

```

1 package pekan9_2511531014;
2
3 public class BTDriver_2511531014 {
4     public static void main(String[] args) {
5         //Membuat pohon
6         BTree_2511531014 tree_1014 = new BTree_2511531014 ();
7         System.out.println("Jumlah simpul awal pohon: ");
8         System.out.println(tree_1014.countNodes_1014());
9         //Menambahkan simpul data 1
10        Node_2511531014 root_1014 = new Node_2511531014 (1);
11        //menjadikan simpul 1 sebagai root
12        tree_1014.setRoot_1014(root_1014);
13        System.out.println("Jumlah simpul jika hanya ada root");
14        System.out.println(tree_1014.countNodes_1014());
15        Node_2511531014 node2_1014 = new Node_2511531014(2);
16        Node_2511531014 node3_1014 = new Node_2511531014(3);
17        Node_2511531014 node4_1014 = new Node_2511531014(4);
18        Node_2511531014 node5_1014 = new Node_2511531014(5);
19        Node_2511531014 node6_1014 = new Node_2511531014(6);
20        Node_2511531014 node7_1014 = new Node_2511531014(7);
21        Node_2511531014 node8_1014 = new Node_2511531014(8);
22        Node_2511531014 node9_1014 = new Node_2511531014(9);
23        root_1014.setLeft_1014(node2_1014);
24        node2_1014.setLeft_1014(node4_1014);
25        node2_1014.setRight_1014(node5_1014);
26        node4_1014.setRight_1014(node8_1014);
27        root_1014.setRight_1014(node3_1014);
28        node3_1014.setLeft_1014(node6_1014);
29        node3_1014.setRight_1014(node7_1014);
30        node6_1014.setLeft_1014(node9_1014);
31        //set root
32        tree_1014.setCurrent_1014(tree_1014.getRoot_1014());
33        System.out.println("menampilkan simpul terakhir:");
34        System.out.println(tree_1014.getCurrent_1014().getData_1014());
35        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
36        System.out.println(tree_1014.countNodes_1014());
37        System.out.println("InOrder:");
38        tree_1014.printInorder_1014();
39        System.out.println("\nPreOrder: ");
40        tree_1014.printInorder_1014();
41        System.out.println("\nPostOrder: ");
42        tree_1014.printInorder_1014();
43        System.out.println("\nDmenampilkan simpul dalam bentuk pohon ");
44        tree_1014.print_1014();
45    }
46 }
47
48 }

```

- Pembahasan

Program ini dibuat dengan mendefinisikan class **BTDriver_2511531014** yang digunakan sebagai program utama untuk menguji implementasi struktur data **Binary Tree** menggunakan class **BTree_2511531014** dan **Node_2511531014**. Program ini berfungsi untuk membuat sebuah pohon biner, menambahkan node, menghitung jumlah simpul, serta menampilkan data pohon menggunakan beberapa metode traversal yaitu InOrder, PreOrder, dan PostOrder.

1. Pada method main(), program membuat objek **BTree_2511531014** dengan nama tree_1014. Objek ini digunakan untuk mengelola struktur pohon biner yang dibuat. Pada awal program dilakukan pengecekan jumlah simpul menggunakan method countNodes_1014() karena kondisi awal pohon masih kosong.
2. Program membuat sebuah node dengan nilai 1 menggunakan class **Node_2511531014**. Node tersebut kemudian dijadikan sebagai root atau simpul utama pada pohon menggunakan method setRoot_1014(). Root merupakan node paling awal yang menjadi pusat dari struktur Binary Tree.
3. Setelah root dibuat, program membuat beberapa node tambahan yaitu node2_1014 sampai node9_1014 yang berisi data angka. Setiap node merepresentasikan sebuah simpul pada Binary Tree yang nantinya akan dihubungkan sebagai anak kiri (left) atau anak kanan (right).
4. Proses pembentukan struktur pohon dilakukan menggunakan method setLeft_1014() dan setRight_1014(). Method tersebut digunakan untuk menghubungkan node satu dengan node lainnya. Contohnya, node 2

dijadikan anak kiri dari root, sedangkan node 3 dijadikan anak kanan dari root.

5. Setelah seluruh node terhubung, program menggunakan method `setCurrent_1014()` untuk menentukan posisi node yang sedang aktif. Pada program ini node terakhir yang diatur adalah root dari tree, kemudian data node tersebut ditampilkan menggunakan method `getData_1014()`.
6. Method `countNodes_1014()` digunakan untuk menghitung jumlah seluruh simpul yang terdapat pada Binary Tree. Perhitungan dilakukan secara rekursif dengan menghitung jumlah node pada bagian kiri dan bagian kanan hingga seluruh node dalam pohon selesai dihitung.
7. Program menampilkan isi Binary Tree menggunakan tiga metode traversal, yaitu:
 - **InOrder**: mengunjungi node kiri, root, kemudian node kanan. Traversal ini biasanya menghasilkan urutan data berdasarkan posisi node pada pohon.
 - **PreOrder**: mengunjungi root terlebih dahulu, kemudian node kiri dan node kanan.
 - **PostOrder**: mengunjungi node kiri dan kanan terlebih dahulu, kemudian root.
8. Method `print_1014()` digunakan untuk menampilkan bentuk struktur pohon secara visual. Method ini membantu melihat hubungan antara parent node dan child node pada Binary Tree.
9. Secara keseluruhan, program ini digunakan untuk memahami konsep dasar struktur data **Binary Tree**, mulai dari pembuatan node, menentukan root, menghubungkan node kiri dan kanan, menghitung jumlah simpul, serta melakukan proses traversal pada sebuah pohon menggunakan bahasa pemrograman Java.

1.4. GraphTraversal_2511531014

- Kode Program

```

1 package pekan9_2511531014;
2
3 import java.util.*;
4 public class GraphTraversal_2511531014 {
5     private Map<String, List<String>> graph_1014 = new HashMap<>();
6
7     // Menambahkan edge (scaf tak kecacah)
8     public void addEdge_1014(String node1_1014, String node2_1014) {
9         graph_1014.putIfAbsent(node1_1014, new ArrayList<>());
10        graph_1014.putIfAbsent(node2_1014, new ArrayList<>());
11        graph_1014.get(node1_1014).add(node2_1014);
12        graph_1014.get(node2_1014).add(node1_1014);
13    }
14
15    // Menampilkan scaf awal
16    public void printGraph_1014() {
17        System.out.println("Graf Awal (Adjacency List):");
18        for (String node_1014 : graph_1014.keySet()) {
19            System.out.print(node_1014 + " -> ");
20            List<String> neighbors_1014 = graph_1014.get(node_1014);
21            System.out.println(String.join(", ", neighbors_1014));
22        }
23        System.out.println();
24    }
25
26    // DFS Rekursif
27    public void dfs_1014(String start_1014) {
28        Set<String> visited_1014 = new HashSet<>();
29        System.out.println("Penelusuran DFS:");
30        dfsHelper_1014(start_1014, visited_1014);
31        System.out.println();
32    }
33
34    private void dfsHelper_1014(String current_1014, Set<String> visited_1014) {
35        if (visited_1014.contains(current_1014)) return;
36        visited_1014.add(current_1014);
37        System.out.print(current_1014 + " ");
38        for (String neighbor_1014 : graph_1014.getOrDefault(current_1014, new ArrayList<>())) {
39            dfsHelper_1014(neighbor_1014, visited_1014);
40        }
41    }
42
43    // BFS Iteratif
44    public void bfs_1014(String start_1014) {
45        Set<String> visited_1014 = new HashSet<>();
46        Queue<String> queue_1014 = new LinkedList<>();
47        queue_1014.add(start_1014);
48        visited_1014.add(start_1014);
49        System.out.println("Penelusuran BFS:");
50        while (!queue_1014.isEmpty()) {
51            String current_1014 = queue_1014.poll();
52            System.out.print(current_1014 + " ");
53            for (String neighbor_1014 : graph_1014.getOrDefault(current_1014, new ArrayList<>())) {
54                if (!visited_1014.contains(neighbor_1014)) {
55                    queue_1014.add(neighbor_1014);
56                    visited_1014.add(neighbor_1014);
57                }
58            }
59        }
60    }
61
62    // Main
63    public static void main(String[] args) {
64        GraphTraversal_2511531014 graph_1014 = new GraphTraversal_2511531014();
65
66        // Contoh scaf : A-B, A-C, B-D, B-E
67        graph_1014.addEdge_1014("A", "B");
68        graph_1014.addEdge_1014("A", "C");
69        graph_1014.addEdge_1014("B", "D");
70        graph_1014.addEdge_1014("B", "E");
71
72        // Scaf awal
73        System.out.println("Graf Awal adalah:");
74        graph_1014.printGraph_1014();
75
76        // Lakukan penelusuran
77        graph_1014.dfs_1014("A");
78        graph_1014.bfs_1014("A");
79    }
80 }

```

• Pembahasan

Program ini dibuat dengan mendefinisikan class **GraphTraversal_2511531014** yang digunakan untuk mengimplementasikan struktur data **Graph** serta melakukan proses penelusuran menggunakan algoritma **DFS (Depth First Search)** dan **BFS (Breadth First Search)**. Program ini menggunakan konsep **Adjacency List** untuk menyimpan hubungan antar node pada graf.

1. Pada class **GraphTraversal_2511531014**, terdapat variabel **graph_1014** yang menggunakan struktur data **HashMap** untuk menyimpan graf. Key pada **HashMap** digunakan sebagai node, sedangkan value berupa **List** yang menyimpan node-node yang terhubung dengan node tersebut.
2. Method **addEdge_1014(String node1_1014, String node2_1014)** digunakan untuk menambahkan hubungan antar dua node pada graf. Karena graf yang dibuat merupakan graf tidak berarah (undirected graph), maka setiap node akan saling terhubung dua arah. Contohnya hubungan A-B akan disimpan sebagai A memiliki tetangga B dan B memiliki tetangga A.

3. Method `printGraph_1014()` digunakan untuk menampilkan bentuk awal graf dalam bentuk **Adjacency List**. Program melakukan perulangan pada setiap node yang terdapat dalam `HashMap` kemudian menampilkan node beserta daftar node yang terhubung dengannya.
4. Method `dfs_1014(String start_1014)` digunakan untuk menjalankan algoritma **DFS (Depth First Search)**. DFS melakukan pencarian dengan cara mengunjungi node sedalam mungkin terlebih dahulu sebelum kembali ke node sebelumnya. Proses DFS menggunakan bantuan method `dfsHelper_1014()` yang bekerja secara rekursif.
5. Pada method `dfsHelper_1014()`, variabel `visited_1014` digunakan untuk menyimpan node yang sudah pernah dikunjungi agar tidak terjadi pengulangan. Program akan mencetak node saat pertama kali dikunjungi kemudian melanjutkan pencarian ke node tetangganya hingga seluruh node selesai ditelusuri.
6. Method `bfs_1014(String start_1014)` digunakan untuk menjalankan algoritma **BFS (Breadth First Search)**. Berbeda dengan DFS, BFS melakukan pencarian berdasarkan tingkat kedekatan node dengan node awal. Algoritma ini menggunakan struktur data **Queue** untuk mengatur urutan node yang akan dikunjungi.
7. Pada proses BFS, node awal dimasukkan ke dalam queue menggunakan `queue_1014.add(start_1014)`. Selanjutnya program mengambil node dari queue menggunakan `poll()`, kemudian memasukkan node tetangga yang belum dikunjungi ke dalam queue hingga seluruh node berhasil ditelusuri.
8. Pada method `main()`, program membuat objek **GraphTraversal_2511531014** dengan nama `graph_1014`. Kemudian dibuat sebuah graf dengan hubungan antar node yaitu:
 - A terhubung dengan B
 - A terhubung dengan C
 - B terhubung dengan D
 - B terhubung dengan E
9. Setelah graf dibuat, program memanggil method `printGraph_1014()` untuk menampilkan struktur graf awal. Kemudian dilakukan proses traversal menggunakan DFS dan BFS dengan titik awal node **A**.
10. Secara keseluruhan, program ini digunakan untuk memahami konsep dasar struktur data **Graph**, cara menyimpan graf menggunakan **Adjacency List**, serta implementasi algoritma pencarian **DFS dan BFS** dalam bahasa

pemrograman Java. DFS digunakan untuk melakukan pencarian secara mendalam, sedangkan BFS digunakan untuk melakukan pencarian berdasarkan level atau jarak terdekat dari node awal.

TUGAS PEKAN 9

BFS dan DFS

1.Kode Program

```
1 package TugasPekan9_2511531014;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.*;
6
7 public class Museum_2511531014 extends JFrame {
8
9
10     Map<String, ArrayList<String>> graph_1014 = new HashMap<>();
11     GraphPanel_1014 panelGraph_1014;
12     JTextArea hasil_1014;
13     JComboBox<String> start_1014;
14     JComboBox<String> goal_1014;
15
16     String[] node_1014 = {
17         "PintuMasuk",
18         "RuangSejarah",
19         "RuangSeni",
20         "RuangBudaya",
21         "RuangArtefak",
22         "Perpustakaan",
23         "Auditorium",
24         "RuangPameran",
25         "TokoSouvenir",
26         "Kantin"
27     };
28
29     HashMap<String,Point> posisi_1014 = new HashMap<>();
30     ArrayList<String> warnaNode_1014 = new ArrayList<>();
31
32     public Museum_2511531014() {
```

```

33 public Museum_2511531014(){
34     setTitle("Peta Museum BFS DFS 2511531014");
35     setSize(900,600);
36     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
37     buatGraph_1014();
38     buatPosisi_1014();
39
40
41     panelGraph_1014 = new GraphPanel_1014();
42     start_1014 = new JComboBox<>(node_1014);
43     goal_1014 = new JComboBox<>(node_1014);
44     start_1014.setSelectedItem("PintuMasuk");
45     goal_1014.setSelectedItem("Auditorium");
46
47
48     JButton bfs_1014 = new JButton("BFS");
49     JButton dfs_1014 = new JButton("DFS");
50     JButton reset_1014 = new JButton("RESET");
51     hasil_1014 = new JTextArea(5,60);
52     hasil_1014.setEditable(false);
53
54
55     bfs_1014.addActionListener(e->{
56         warnaNode_1014.clear();
57         ArrayList<String> hasil =
58             BFS_1014(
59                 start_1014.getSelectedItem().toString(),
60                 goal_1014.getSelectedItem().toString()
61             );
62         warnaNode_1014.addAll(hasil);
63         hasil_1014.setText(
64             "Metode : BFS\n"+
65             "Jalur : "+hasil+
66             "\nJumlah Node Dieksplorasi : "+hasil.size()
67         );

```

```

68         panelGraph_1014.repaint();
69     });
70     dfs_1014.addActionListener(e->{
71         warnaNode_1014.clear();
72         ArrayList<String> hasil =
73             DFS_1014(
74                 start_1014.getSelectedItem().toString(),
75                 goal_1014.getSelectedItem().toString()
76             );
77         warnaNode_1014.addAll(hasil);
78         hasil_1014.setText(
79             "Metode : DFS\n"+
80             "Jalur : "+hasil+
81             "\nJumlah Node Dieksplorasi : "+hasil.size()
82         );
83         panelGraph_1014.repaint();
84     });
85     reset_1014.addActionListener(e->{
86         warnaNode_1014.clear();
87         hasil_1014.setText("");
88         panelGraph_1014.repaint();
89     });
90
91     JPanel atas_1014 = new JPanel();
92     atas_1014.add(new JLabel("Start"));
93     atas_1014.add(start_1014);
94     atas_1014.add(new JLabel("Goal"));
95     atas_1014.add(goal_1014);
96     atas_1014.add(bfs_1014);
97     atas_1014.add(dfs_1014);
98     atas_1014.add(reset_1014);
99
100     add(atas_1014,BorderLayout.NORTH);
101     add(panelGraph_1014,BorderLayout.CENTER);
102     add(new JScrollPane(hasil_1014),BorderLayout.SOUTH);

```

```

103         setVisible(true);
104     }
105
106     void buatGraph_1014(){
107         for(String x:node_1014)
108             graph_1014.put(x,new ArrayList<>());
109
110         tambah_1014("PintuMasuk","RuangSejarah");
111         tambah_1014("PintuMasuk","RuangSeni");
112         tambah_1014("RuangSejarah","RuangBudaya");
113         tambah_1014("RuangSejarah","Perpustakaan");
114         tambah_1014("RuangSeni","RuangArtefak");
115         tambah_1014("RuangSeni","Auditorium");
116         tambah_1014("RuangBudaya","RuangPameran");
117         tambah_1014("RuangBudaya","TokoSouvenir");
118         tambah_1014("RuangArtefak","RuangPameran");
119         tambah_1014("RuangArtefak","Perpustakaan");
120         tambah_1014("Perpustakaan","Auditorium");
121         tambah_1014("Auditorium","Kantin");
122         tambah_1014("RuangPameran","TokoSouvenir");
123         tambah_1014("TokoSouvenir","Kantin");
124         tambah_1014("RuangPameran","Kantin");
125     }
126
127     void tambah_1014(String a,String b){
128         graph_1014.get(a).add(b);
129         graph_1014.get(b).add(a);
130     }
131
132     ArrayList<String> BFS_1014(String start,String goal){
133         Queue<ArrayList<String>> q = new LinkedList<>();
134         ArrayList<String> awal = new ArrayList<>();
135         awal.add(start);
136         q.add(awal);
137         HashSet<String> visit = new HashSet<>();
138         while(!q.isEmpty()){
139             ArrayList<String> jalan=q.poll();

```

```

137         ArrayList<String> jalan=q.pop();
138         String akhir=jalan.get(jalan.size()-1);
139         if(akhir.equals(goal))
140             return jalan;
141         if(!visit.contains(akhir)){
142             visit.add(akhir);
143             for(String next:graph_1014.get(akhir)){
144                 ArrayList<String> baru =
145                     new ArrayList<>(jalan);
146                 baru.add(next);
147                 q.add(baru);
148             }
149         }
150     }
151     return new ArrayList<>();
152 }
153
154 ArrayList<String> DFS_1014(String start,String goal){
155     Stack<ArrayList<String>> stack=new Stack<>();
156     ArrayList<String> awal=new ArrayList<>();
157     awal.add(start);
158     stack.push(awal);
159     HashSet<String> visit=new HashSet<>();
160     while(!stack.empty()){
161         ArrayList<String> jalan=stack.pop();
162         String akhir=jalan.get(jalan.size()-1);
163         if(akhir.equals(goal))
164             return jalan;
165         if(!visit.contains(akhir)){
166             visit.add(akhir);
167             for(String next:graph_1014.get(akhir)){
168                 ArrayList<String> baru =
169                     new ArrayList<>(jalan);
170                 baru.add(next);
171                 stack.push(baru);
172             }

```

```

172             stack.push(baru);
173         }
174     }
175     return new ArrayList<>();
176 }
177 void buatPosisi_1014(){
178     posisi_1014.put("PintuMasuk",new Point(400,50));
179     posisi_1014.put("RuangSejarah",new Point(400,130));
180     posisi_1014.put("RuangSeni",new Point(250,180));
181     posisi_1014.put("RuangBudaya",new Point(400,230));
182     posisi_1014.put("RuangArtefak",new Point(600,180));
183     posisi_1014.put("Perpustakaan",new Point(250,330));
184     posisi_1014.put("Auditorium",new Point(600,330));
185     posisi_1014.put("RuangPameran",new Point(400,330));
186     posisi_1014.put("TokoSouvenir",new Point(300,430));
187     posisi_1014.put("Kantin",new Point(550,430));
188 }
189 class GraphPanel_1014 extends JPanel{
190     protected void paintComponent(Graphics g){
191         super.paintComponent(g);
192         g.setFont(new Font("Arial",Font.PLAIN,12));
193         for(String a:graph_1014.keySet()){
194             for(String b:graph_1014.get(a)){
195                 Point p1=posisi_1014.get(a);
196                 Point p2=posisi_1014.get(b);
197                 g.drawLine(
198                     p1.x,p1.y,
199                     p2.x,p2.y
200                 );
201             }
202         }
203     }

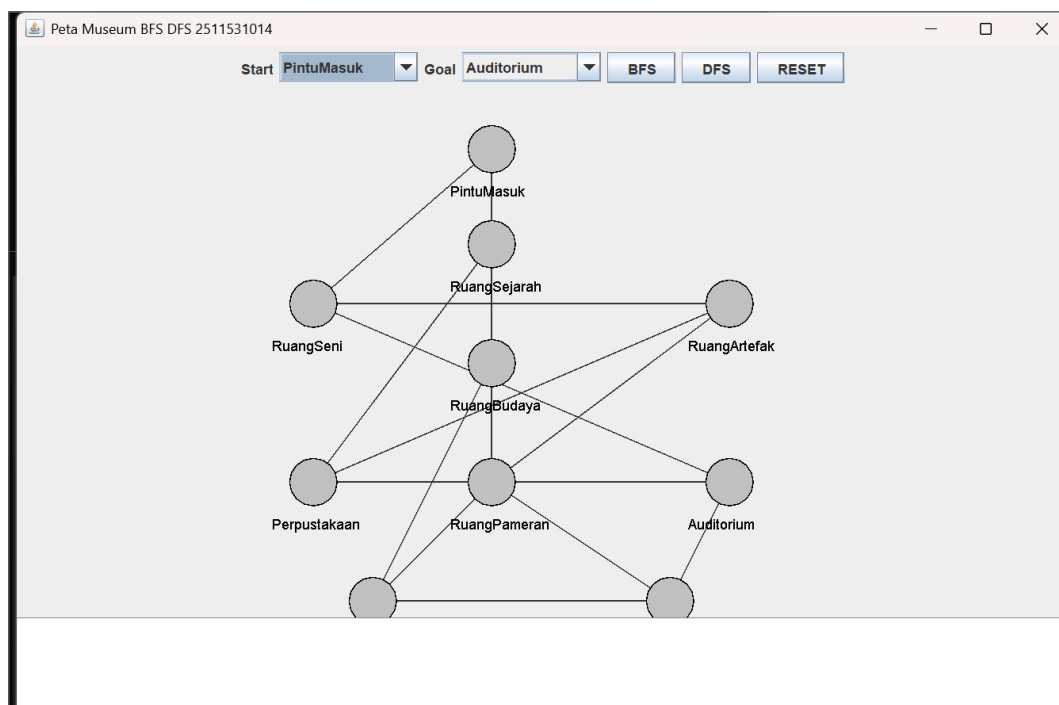
```

```

204     for(String x:node_1014){
205         Point p=posisi_1014.get(x);
206         if(warnaNode_1014.contains(x))
207             g.setColor(Color.GREEN);
208         else
209             g.setColor(Color.LIGHT_GRAY);
210         g.fillOval(
211             p.x-20,
212             p.y-20,
213             40,
214             40
215         );
216         g.setColor(Color.BLACK);
217         g.drawOval(
218             p.x-20,
219             p.y-20,
220             40,
221             40
222         );
223         g.drawString(
224             x,
225             p.x-35,
226             p.y+40
227         );
228     }
229 }
230 }
231 public static void main(String[] args){
232     new Museum_2511531014();
233 }
234 }

```

2. Output Program



3. Deskripsi singkat

Peta yang dibuat adalah peta sederhana **Museum** yang terdiri dari beberapa lokasi penting yang terdapat di dalam area museum. Lokasi-lokasi tersebut direpresentasikan sebagai simpul (**node**) dalam graph, sedangkan jalur penghubung antar lokasi direpresentasikan sebagai sisi (**edge**).

Node yang digunakan yaitu:

- o PintuMasuk
- o RuangSejarah
- o RuangSeni
- o RuangBudaya
- o RuangArtefak
- o Perpustakaan
- o Auditorium
- o RuangPameran
- o TokoSouvenir
- o Kantin

Graph ini digunakan untuk mencari jalur dari lokasi awal (**Start**) menuju lokasi tujuan (**Goal**) menggunakan algoritma **BFS (Breadth First Search)** dan **DFS (Depth First Search)**.

Jumlah node dan edge

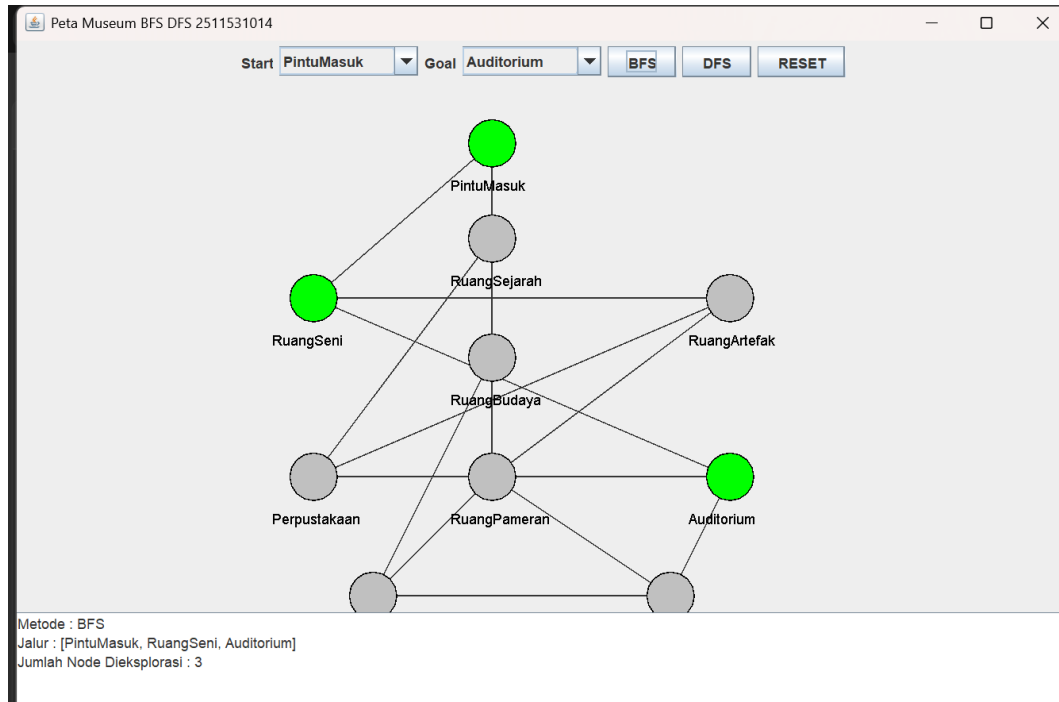
- o **Jumlah Node (Vertex) : 10 Node**
- o **Jumlah Edge (Sisi) : 15 Edge**

o Daftar Edge

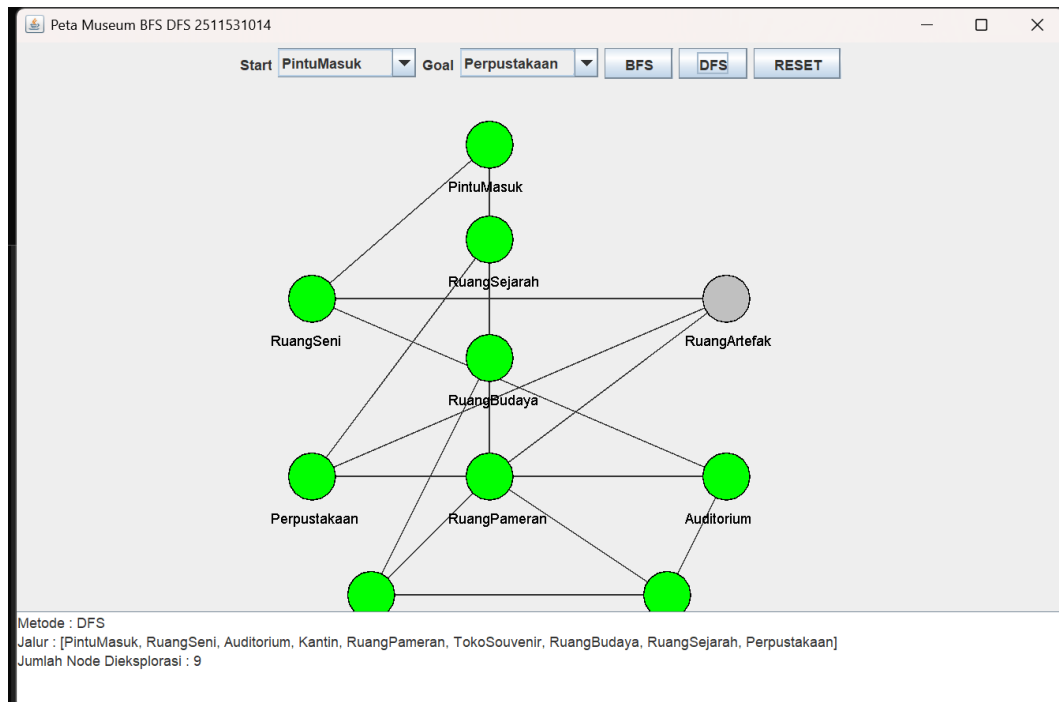
- PintuMasuk – RuangSejarah
- PintuMasuk – RuangSeni
- RuangSejarah – RuangBudaya
- RuangSejarah – Perpustakaan
- RuangSeni – RuangArtefak
- RuangSeni – Auditorium
- RuangBudaya – RuangPameran
- RuangBudaya – TokoSouvenir
- RuangArtefak – RuangPameran
- RuangArtefak – Perpustakaan
- Perpustakaan – Auditorium
- Auditorium – Kantin
- RuangPameran – TokoSouvenir

- TokoSouvenir – Kantin
- RuangPameran – Kantin

4. BFS



5.DFS



5. Kesimpulan

Kesimpulan perbandingan BFS dan DFS

Berdasarkan hasil pengujian, algoritma **BFS (Breadth First Search)** dan **DFS (Depth First Search)** memiliki cara kerja yang berbeda dalam melakukan pencarian jalur pada graph Museum.

- o **BFS** melakukan pencarian secara melebar (per level), sehingga dapat menemukan jalur dengan jumlah langkah paling sedikit dari lokasi awal menuju lokasi tujuan.

- o **DFS** melakukan pencarian secara mendalam terlebih dahulu dengan menelusuri satu jalur sampai menemukan tujuan.

- o **BFS** biasanya mengunjungi lebih banyak node karena harus memeriksa kemungkinan jalur pada setiap tingkat sebelum melanjutkan ke tingkat berikutnya.

- o **DFS** cenderung menggunakan memori lebih sedikit, tetapi jalur yang ditemukan belum tentu merupakan jalur terpendek karena bergantung pada urutan node yang diperiksa.

- o Pada pengujian ini, BFS menghasilkan jalur yang lebih pendek dari **PintuMasuk menuju Auditorium**, sedangkan DFS menghasilkan jalur berbeda sesuai dengan urutan penelusuran node pada graph.

Kesimpulannya, BFS lebih cocok digunakan ketika tujuan utama adalah mendapatkan jalur terpendek pada peta Museum, sedangkan DFS lebih cocok digunakan untuk melakukan pencarian secara mendalam ketika tidak terlalu memperhatikan panjang jalur yang ditemukan.

